PiPER 机械臂视觉抓取 · 完整流程文档

> 目标：让 AgileX **PiPER** 六轴机械臂，用装在夹爪上方的 **Orbbec Dabai DC1** 深度相机，识别桌面上的方盒并自动抓取。 文档建立：2026-11-04 · 对应实测固件 S-V1.9-0 · 全部命令在本机（CachyOS, PubliCachy）验证过。

---

0. 全景：这件事为什么不能"一步到位"

从"相机看见一个盒子"到"夹爪夹住它"，中间隔着一整条坐标链。少任何一环，机械臂就只能瞎动：

```
像素 (u,v) + 深度 d
      │  ① 相机内参 K + 畸变 dist
      ▼
相机坐标系 P_cam   (mm, 光学中心为原点, Z 向前)
      │  ② 手眼矩阵 T_flange→cam   ← 通过"手眼标定"求
      ▼
法兰坐标系 P_flange
      │  ③ 正运动学 (控制器直接给) T_base→flange
      ▼
基座坐标系 P_base  (mm, 机械臂底座为原点)
      │  ④ 抓取几何: 抬高 80mm、姿态竖直向下、夹爪开度
      ▼
一串 move_p / grip 指令
```

- ①③ 是"标定 + 读状态"，好办。
- ② **是这件事的真正门槛**：相机是跟着夹爪一起动的（eye-in-hand），它看到的世界每帧都在变，必须求出一个**固定不变**的 4×4 矩阵 `T_flange→cam`，把"相机眼里的点"翻译成"机械臂脚下的点"。这个矩阵靠**手眼标定**（解 AX = XB）得到。
- ④ 是策略：抓多高、从哪进、夹多宽、用多大力。

下面按"接线 → 环境 → 控制 → 视觉 → 标定 → 抓取"的顺序展开。

---

1. 硬件与链路

部件	型号 / 参数	备注
机械臂	AgileX **PiPER**，6 关节 + 平行夹爪	固件 **S-V1.9-0**，硬件 H-V1.2-1，节点 ARM_MC
通信	USB-CAN 适配器 candleLight（`1d50:606f`, `gs_usb`）→ `can0`	**1 Mbps 固定**，关节反馈 **200 Hz**
深度相机	**Orbbec Dabai DC1**	`2bc5:0557` 彩色(UVC) + `2bc5:0657` 深度(厂商接口)，SN CC1N16203VB，USB 2.0
相机安装	夹爪上方，**eye-in-hand**，**倒装**	图像必须 `--rotate 180` 才是正的

关节限位（度）：J1 ±150 · J2 0~180 · J3 −170~0 · J4 ±100 · J5 ±69.9 · J6 ±120；夹爪行程 0~70 mm。

1.1 上电自检（三条命令）

```
lsusb | grep -E "1d50|2bc5"          # 应看到 CAN 适配器 + 两个 Orbbec 设备
ip -details link show can0           # state UP, bitrate 1000000
systemctl status can0-piper          # active (running)
```

CAN 已做成**插拔自动拉起**，不需要每次手动配：

- `/etc/systemd/system/can0-piper.service` — 先把 can0 拉 down，再设 1 Mbps，再 up
- `/etc/udev/rules.d/80-can-piper.rules` — 检测到 usb-can 插入就 SYSTEMD_WANTS 上面的服务
- 手动重来：`sudo /home/po/arm/can_up.sh`

相机侧 udev：`/etc/udev/rules.d/99-obsensor-libusb.rules` 给 `2bc5:*` 设备 0666 权限，并建符号链接 `/dev/dabai_dc1`（深度）、`/dev/dabai_dc1_rgb`（彩色）。不用 sudo 就能开相机。

---

2. 软件环境（为什么是两个 venv）

环境	Python	内容	用途
`/home/po/arm-env`	3.13	**pyAgxArm 1.0.0**、piper_sdk 0.6.2、python-can	机械臂控制
`/home/po/vision-env`	**3.11**	numpy 1.26.4、opencv-python-headless 4.11、**pyorbbecsdk**（源码编译）	相机 / 视觉 / 标定

两个环境分开是**被迫的**，不是设计选择：

1. **为什么用 pyAgxArm 而不是 piper_sdk** 官方《PIPER 产品使用资料清单》把二次开发分成两组：
2. 新组（兼容**所有**固件）：`pyAgxArm`、`agx_arm_ros`、`agx_arm_urdf`、`agx_arm_sim`
3. 旧组：`piper_sdk`、`piper_ros`，**只兼容 S-V1.8-8 以下** 这台是 **S-V1.9-0**，超出旧组范围 → 只能用 pyAgxArm（驱动 profile `PiperFW.V189`）。旧的 `arm_piper_sdk_legacy.py` 保留仅作参考，**别在 1.9 固件上拿它做控制**。 pyAgxArm 不在 PyPI，要装 git 版：`uv pip install "git+https://github.com/agilexrobotics/pyAgxArm.git"`。
4. **为什么 pyorbbecsdk 必须自己编译** PyPI 上 `pyorbbecsdk 1.3.2` 的 manylinux wheel 解开一看是 **macOS 二进制**（`pyorbbecsdk.cpython-311-darwin.so` + `libOrbbecSDK.dylib`），Linux 上装完 import 直接失败。 于是从官方仓库源码编译，而且**分支必须是 `main`（SDK v1.x, 1.10.16）**——默认的 `v2-main` 是 SDK 2.x，**不支持 Dabai DC1**。 产物（`.so` + `libOrbbecSDK.so.1.10.16`）已拷进 venv 的 site-packages，并用 `patchelf --set-rpath '$ORIGIN'` 去掉指向源码目录/临时目录的 RUNPATH，所以可以脱离源码目录运行。 只能

Python 3.11：官方只给了 cp311 的 pybind11 绑定。

5. 顺带：numpy 必须 <2.0（SDK v1 的要求），3.13/3.14 没有可用绑定。

---

3. 机械臂控制：API 与两个致命坑

3.1 最小连接代码

```
from pyAgxArm import create_agx_arm_config, AgxArmFactory, ArmModel, PiperFW
cfg = create_agx_arm_config(robot=ArmModel.PIPER,
                            firmeware_version=PiperFW.V189,   # 拼写就是 firmeware
                            channel="can0")
arm = AgxArmFactory.create_arm(cfg)
arm.connect()
```

常用接口：

分类	接口	说明
读	`get_joint_angles()`	6 关节弧度（`.msg` / `.hz`）
读	`get_flange_pose()`	法兰位姿：x,y,z (m) + rx,ry,rz (rad 欧拉角)
读	`get_arm_status().msg`	ctrl_mode / arm_status / motion_status / teach_status / trajectory_num / err_status
控	`enable()` / `disable()`	使能（上劲保持）/ 失能（**会掉**）
控	`set_speed_percent(0~100)`	全局速度百分比
控	`move_j([6 rad])`	关节空间运动

分类	接口	说明
控	`move_p([x,y,z,rx,ry,rz])`	笛卡尔位姿（m + rad）
控	`init_effector(OPTIONS.EFFECTOR.AGX_GRIPPER)` → `move_gripper_m(value=米, force=N)`	夹爪
危险	`reset()` / `calibrate_joint()`	reset 立刻断电掉臂；calibrate_joint 会把当前位置设成零点

> `calibrate_joint(1..6 | 255)` 是**改零点**，不是"标定"。当前零点正常，**永远不要手滑调用**。

3.2 坑一：单帧指令不生效

发一次 `move_j` 控制器**不理**。它要求目标帧持续流入，实测必须连续发约 **0.8 秒**（≥20 帧）才开始动。所以脚本里是这种写法：

```
t_end = time.time() + 0.8
while time.time() < t_end:
    arm.move_j(target_rad)
    time.sleep(0.02)
# 然后轮询 motion_status 直到 0
```

3.3 坑二：第一帧被丢掉

上电后 `ctrl_mode = TEACHING_MODE(0x2)`（拖动示教态）。第一次发运动指令时，控制器**先把模式切到 CAN_CTRL(0x1)，切换那一帧的目标被丢弃**。表现就是"第一次发指令看起来没反应"，第二次才开始动。判断方法：发之前读一次 `ctrl_mode`，是 0x2 就预期第一帧无效。

3.4 坑三：J5 的角度增大会让视角"抬高"（本机实测）

这台臂装好相机后的几何关系与直觉相反。实测 J5：16.55° → 21.61°（+15°）时视野**从平视房间变成看到桌面**，而 −10° 反而抬到天花。**先用小角度试方向，别按直觉连发大角度。**

4. 视觉：取图与检测

4.1 取一对 RGB-D

```
/home/po/vision-env/bin/python /home/po/vision/capture.py --rotate 180 --tag desk
```

输出到 `/home/po/vision/out/`：

- `<tag>_color.png` — 彩色 640×480
- `<tag>_depth.npy` — uint16，单位 mm，**已用 AlignFilter 对齐到彩色**（像素一一对应）
- `<tag>_depth_vis.png` — 伪彩（0~3000 mm）

关键实现点：

- 彩色是 MJPG 流，要用 `cv2.imdecode` 解，不能直接当 BGR 用
- 深度原始 640×400（Y11），对齐到彩色后变 640×480，此后 (u,v) 可直接套彩色内参
- 倒装所以整幅旋转 180°（旋转后内参 cx/cy 要相应变：`cx' = W-1-cx`, `cy' = H-1-cy`）

4.2 深度相机的脾气（必须知道）

- **深度空洞严重**：室内 3~4 m 的墙基本测不到，有效像素只有约 15%；**0.3~1.5 m 是舒适区**，抓取目标应控制在这个距离。
- 反光、透明、纯黑表面深度直接为 0。
- SDK 报的彩色内参 `fx=fy=488.83, cx=314.65, **cy=214.66**` 可疑：640×480 的中心应是 240，214.66 正好是按 640×400 算的。**所以内参一定要自己标**，别信默认值。

4.3 检测方盒（`detect_box.py`）

思路是"颜色 + 深度"双门限，不依赖深度学习：

1. 彩色转 HSV，取棕色纸箱区间 `H 8~40, S 40~210, V 60~235`

2. 深度门限 250~1500 mm（挡掉远处的椅子/墙）
3. 形态学开闭 + 连通域，按 **面积 / 长宽比 / 实心度** 过滤
4. 对最优轮廓取 `minAreaRect`（得到朝向 yaw 与外接尺寸）
5. 轮廓内深度取**中位数**（抗噪），用内参反投影成相机系 3D 点
6. 输出 `<tag>_detect.png` 可视化

踩过的坑：腕部平视时，算法会抓到 1.24 m 外的木椅（面积 8459、长宽比 1.1、实心度 0.4），因为"棕色 + 在深度范围内"两条都满足。教训是**视角必须先摆正，让目标占据画面主体**，否则颜色检测一定会被背景骗。

当前实测结果（J5 = 41.64°，相机正对桌面）：

```
像素 (405, 189)   depth 409 mm
相机系 P_cam = (75.6, -21.1, 409.0) mm
尺寸 ≈ 102 × 63 mm   yaw 0°   solidity 0.86   area 7896 px
```

---

5. 标定（这一节是全流程的核心）

标定分两步，**顺序不能反**：先内参（相机自己的几何），再手眼（相机和法兰的相对关系）。

5.1 需要什么靶

黑白棋盘格（OpenCV `findChessboardCorners`），本机已生成 A4 版：`calib/chessboard_9×6_25mm.pdf`（9×6 内角点，25 mm 方格，300 dpi）。

三个硬要求：

1. **必须真的黑白相间**。实测反例：实验室的黑色自愈切割垫（细白网格线）→ 6 种内角点尺寸全部检测失败，只能当垫子用。
2. 打在**平面**上，打印时选"实际大小 100%"，打完**用尺子量一格**是不是 25 mm（打印机缩放很常见），实测值要传给脚本 `--square`。

3. 标定全程**靶不能动**，动的是机械臂。

没有打印机时的替代：在平板上全屏显示 `calib/chessboard_tablet_9×6_25mm.png`（板面 250×175 mm），亮度拉满、避开灯管镜面反光，同样要量一格实际尺寸。

5.2 标定前：把相机摆成俯视

相机装在夹爪上方且倒装，出厂姿态是平视房间，靶根本不在视野里。用 `arm_move.py` 小步压腕（每次 ≤20°，速度 10%，每步读状态）：

```
/home/po/vision/arm_move.py joint J5 20      # 绝对别一次给大角度
/home/po/vision-env/bin/python /home/po/vision/capture.py --rotate 180 --tag t1
```

本机走过的实际链路：`J5 16.55° → 6.58°（视野抬高，确认符号）→ 21.61°（桌面出现）→ 41.64°（正对桌面，盒子完整）`。判据：**靶/目标完整在画面内、离中心不太远、深度有效像素占比明显上升**。

5.3 第一步：内参标定

```
# 拿着板子在相机前摆 15~20 个不同角度（倾斜、远近、偏心），逐个拍
/home/po/vision-env/bin/python /home/po/vision/calibrate_intrinsic.py
```

判据：

- **重投影误差 RMS < 0.5 px** 才算可用；> 1 px 说明图糊了/靶不平/样本姿态太单一
- 姿态要**多样**：板子占画面 1/2~1/4、绕三个轴都有 ±20° 以上的倾斜；全是一模一样的正对姿态 → 内参病态
- 产出 `out/camera_intrinsics.npz`（K, dist, rms）

5.4 第二步：手眼标定（eye-in-hand, AX = XB）

原理一句话：让机械臂走 N 个不同位姿，每个位姿同时记录 **(a) 法兰在基座系的位姿** 和 **(b) 靶在相机系的位姿**。任意两个位姿之间，法兰的相对运动 A 和相机的相对运动 B 满足 `A·X = X·B`，X 就是要求的 `T_flange→cam`。

```
# 1）采数据：臂自动走 16 个位姿（J1±6、J2±4、J3±4、J4±10、J5±6、J6±12 组合，速度 8%）
#    每位姿：等稳 → 拍图 → 立刻读法兰位姿（不要延迟读！）
/home/po/arm-env/bin/python /home/po/vision/handeye.py collect --poses 15 --square 25

# 2）解算：先 calibrateCamera 复核内参，再逐图 solvePnP，最后 calibrateHandEye
/home/po/arm-env/bin/python /home/po/vision/handeye.py solve --square 25
```

脚本做的事：

1. `collect`：以当前姿态为基准叠加小扰动（保证靶始终在视野里），每一位姿存图 + 法兰位姿 → `calib/handeye_data.npz`；结束**回到原姿态**
2. `solve`：`calibrateCamera`（若已有内参则复用）→ 每张图 `solvePnP` 得 `T_target→cam` → `calibrateHandEye` 用 **TSAI / PARK / HORAUD / ANDREFF 四种算法**各解一次
3. 产出 `out/hand_eye.npz`（R, t, 4×4 T）

判据（很实用）：

- 四种算法解出的**平移量应互相接近（差 < 10 mm）**；差异巨大 = 位姿集合退化（旋转轴太单一）
- 平移量量级应在几十毫米（相机到法兰的物理距离），算出 200 mm 或 5 mm 都要怀疑
- `solve` 会打印 `valid pairs`，比采集数少就说明有图没检出棋盘格，需要补拍

常见失败原因：靶中途被碰 → 直接废；位姿只在同一个平面平移没有转动 → 退化；光照过曝/过暗 → 角点亚像素不准。

5.5 第三步：验证（别跳过）

只动不抓的预演，这是最省钱的安全网：盒子放回原位 → 相机拍一帧 → 算出抓取点 → `move_p` 走到盒子上方 **80 mm** 停住。人眼确认夹爪正对盒子，再执行下降。误差大就说明手眼不对，回去查位姿多样性，不要硬抓。

---

6. 抓取执行

6.1 数学（一行都不能省）

```
P_cam  = ((u-cx)*d/fx, (v-cy)*d/fy, d)        # d 用轮廓内深度中值
P_base = T_base→flange(t_snapshot) · T_flange→cam · P_cam
approach = [P_base.x, P_base.y, P_base.z + 80mm, R_top_down]
grasp    = [P_base.x, P_base.y, P_base.z + 盒高/2, R_top_down]
```

三个细节：

- `T_base→flange` 必须用**拍照那一瞬间**的法兰位姿；拍完再动再读，误差就是整段位移
- `R_top_down`（夹爪竖直向下）不要现算：手动把腕部摆到竖直向下，读一次欧拉角存成常量
- 夹爪闭合宽度取 **盒宽 − 10 mm**，抓取高度取 **盒顶 + 半个盒高**（用盒底会撞桌面）

6.2 动作序列（每步都检查）

步	动作	检查
1	张开夹爪到 盒宽−10 mm	开度反馈正常
2	`move_p` → approach（盒上方 80 mm）	`motion_status=0`、无 err
3	`move_p` → grasp（直线下降）	同上；深度/逆解异常立即 abort
4	闭合夹爪（力 1~2 N），停 0.3 s	开度反馈 ≈ 盒宽 → 抓到了
5	`move_p` → 抬起 80 mm	同上
6	（可选）移到放置点，张开释放	—

全程 `set_speed_percent(10)`；每步"发 0.8 s 目标帧 + 等 `motion_status` 回 0"再走下一步。

6.3 一个绕不过去的物理问题

相机装在夹爪**上方**，所以下降到最后几十毫米时，**夹爪本身会挡住视线**（当前画面底部那团黑色

就是）。所以抓取是**开环**的：在 approach 位姿把目标算准 → 后面按计算值下降到闭合，不再依赖视觉。approach 高度（80 mm）就是为了"还在看得见的时候定好位"。

---

7. 安全规矩（照做，别试）

1. **PiPER 没有抱闸，断电会掉。** `disable`、拔线、`reset()` 之前先用手托住末端。
2. 第一次试任何运动：**速度 10~15%、角度 5~10°**，先验证方向对不对。
3. 关节限位会被 SDK 拦，但**桌面/自身碰撞不会被拦** —— Z 别低于约 20 mm，别让腕部翻到手够不着的地方。
4. 通电时**不要热插拔 CAN 线**，会打断 200 Hz 反馈。
5. 真急停按机械臂上的**物理按钮**；`arm.py estop` 是软件急停（阻尼缓降，事后要重新 enable）。
6. 抓取前确认：工作区内没人手、桌上没有易倒物、盒子重量在夹爪力（1~2 N，可调）能力内。

---

8. 命令速查

```
# ---- 机械臂（/home/po/arm/arm.py，pyAgxArm）----
/home/po/arm/arm.py status                          # 只读：状态/关节/法兰
/home/po/arm/arm.py monitor                         # 10 Hz 实时刷新
/home/po/arm/arm.py enable --speed 20               # 使能（会保持当前姿态）
/home/po/arm/arm.py joint 5 0 0 0 0 0 --speed 10    # 关节角（度）
/home/po/arm/arm.py movep 150 0 250 180 0 0 --speed 10
/home/po/arm/arm.py grip 35 1                       # 夹爪 35 mm / 1 N
/home/po/arm/arm.py stop                            # 平滑停住并保持
/home/po/arm/arm.py disable                         # 失能（先托住！）

# ---- 视觉 ----
/home/po/vision-env/bin/python /home/po/vision/capture.py --rotate 180 --tag t1
/home/po/vision-env/bin/python /home/po/vision/detect_box.py --tag t1
```

```
/home/po/vision-env/bin/python /home/po/vision/calibrate_intrinsic.py
/home/po/arm-env/bin/python    /home/po/vision/handeye.py collect --poses 15 --square 25
/home/po/arm-env/bin/python    /home/po/vision/handeye.py solve --square 25

# ---- 小步助手 ----
/home/po/vision/arm_move.py joint J5 10          # 只动 J5 ±10°（限 ±25°）
/home/po/vision/arm_move.py lift 20              # 沿 +Z 抬 20 mm
```

---

9. 排错表

现象	原因	处理
`can0` 不存在 / DOWN	USB-CAN 没插好 / 服务没起	`lsusb | grep 1d50`；`sudo /home/po/arm/can_up.sh`
发指令机械臂不动	单帧指令 / 首次切 ctrl_mode 丢帧	连发 0.8 s；读 `ctrl_mode` 确认已切 CAN_CTRL
动一下就报错停	逆解失败（超出可达空间/奇异位姿）	换中间位姿绕开，别硬顶
`import pyorbbecsdk` 失败	装了 PyPI 的 macOS wheel	用 `/home/po/vision-env`，别 reinstall 覆盖
相机打不开（权限）	udev 规则未生效	重插设备；`ls -l /dev/dabai_dc1*` 应为 0666
深度大片 0	目标太远/反光/纯黑	目标移到 0.3~1.2 m；换角度避开高光
检测抓到背景家具	视角太平时背景进入深度带	先压腕俯视，让目标占画面主体
棋盘格检测失败	靶不是黑白方格（如切割垫）	打 A4 棋盘格或平板全屏显示
内参 rms > 1 px	样本姿态单一 / 靶不平 / 图糊	补多角度样本，重拍

现象	原因	处理
四种手眼算法结果差很大	位姿旋转轴单一（退化）	加绕不同轴的转动，重采

---

10. 进度与下一步

已完成

- 硬件链路：CAN 1 Mbps 自动拉起、相机 udev、两个 venv（pyAgxArm 3.13 / pyorbbecsdk 3.11 源码编译）
- 控制：`arm.py`（status/monitor/enable/joint/movep/grip/stop/estop）、`arm_move.py`（小步安全动）
- 视觉：`capture.py` 取对齐 RGB-D、`detect_box.py` 检出方盒并给相机系 3D（当前 409 mm / 102×63 mm）
- 视角：J5 已压到 41.64°，相机正对桌面，目标完整可见
- 脚本：`handeye.py`（采集 + 解算，支持 `--square`）、`calibrate_intrinsic.py`

待完成（按顺序）

1. **拿到可用棋盘格靶**（打印 A4 或用平板全屏），量一格实测尺寸
2. 跑内参复核 → 判据 rms < 0.5 px
3. 跑手眼标定 → 判据四种算法平移差 < 10 mm
4. 只动不抓的 approach 预演 → 人眼确认夹爪对准盒子
5. 执行抓取（速度 10%，逐级下降）→ 闭合 → 抬起
6. 可选：盒子放不同位置验证鲁棒性；把整链封装成 `grasp.py` 一条命令

---

11. 文件清单

```
/home/po/arm/
├── arm.py                          # pyAgxArm 命令行（主用）
├── arm_piper_sdk_legacy.py         # 旧固件版（仅参考）
├── can_up.sh                       # 手动拉起 can0
└── README.md                       # 机械臂笔记（更详细的控制/示教部分）

/home/po/vision/
├── capture.py                      # 取 RGB-D（对齐全彩 + 旋转）
├── detect_box.py                   # 方盒检测 → 相机系 3D
├── arm_move.py                     # 小步安全运动助手
├── calibrate_intrinsic.py          # 棋盘格内参
├── handeye.py                      # 手眼标定（collect / solve）
├── pyorbbecsdk/                    # SDK 源码（main 分支 = v1.10.16）
├── calib/chessboard_9×6_25mm.pdf            # A4 打印用
├── calib/chessboard_tablet_9×6_25mm.png     # 平板全屏用
├── calib/handeye_data.npz          # 标定采样（生成后）
├── out/                            # 所有图与 .npz 输出
├── GRASP_PLAN.md                   # 抓取方案（简版）
└── PIPER_抓取全流程.md              # 本文档

/home/po/arm-env/    等价于 pyAgxArm 环境（python 3.13）
/home/po/vision-env/ 视觉环境（python 3.11）
```